

let's now go deep into Doubly Linked List (DLL) in Java, with:

- 💡 Complete concepts
- ⚙️ All operations (insert, delete, traverse, search)
- 📄 Key notes & important recall points
- 💻 Example code

Perfect 👍 Let's go **deep into Doubly Linked List (DLL) in Java** — covering *all concepts, operations, important notes, and example code* for complete understanding.

💡 Concept of Doubly Linked List (DLL)

A **Doubly Linked List** is a type of **Linked List** in which each node has **three parts**:

1. **Data** – stores the actual information.
2. **Prev** – reference to the previous node.
3. **Next** – reference to the next node.

Unlike a **Singly Linked List**, a DLL allows **traversal in both directions** (forward and backward).

🗨️ Structure of a Doubly Linked List Node

java

📋 Copy code

```
class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

```
}  
}
```

⚙️ Core Operations in DLL

We'll go through all important operations one by one with example code snippets.

1 Insertion Operations

a. Insert at the Beginning

- Create a new node.
- Set its `next` to the current head.
- Update the previous head's `prev` to the new node.
- Move `head` to the new node.

java

 Copy code

```
void insertAtBeginning(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
        return;  
    }  
    newNode.next = head;  
    head.prev = newNode;  
    head = newNode;  
}
```

b. Insert at the End

- Traverse to the last node.
- Set the new node's `prev` to last node and last node's `next` to new node.

java

 Copy code

```
void insertAtEnd(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
    }
```

```
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.next = newNode;
    newNode.prev = temp;
}
```

c. Insert After a Given Node

java

 Copy code

```
void insertAfter(Node prevNode, int data) {
    if (prevNode == null) {
        System.out.println("Previous node cannot be null");
        return;
    }
    Node newNode = new Node(data);
    newNode.next = prevNode.next;
    prevNode.next = newNode;
    newNode.prev = prevNode;
    if (newNode.next != null)
        newNode.next.prev = newNode;
}
```

2 Deletion Operations

a. Delete from Beginning

java

 Copy code

```
void deleteFromBeginning() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    head = head.next;
```

```
    head.prev = null;
}
```

b. Delete from End

java

 Copy code

```
void deleteFromEnd() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.prev.next = null;
}
```

c. Delete by Value

java

 Copy code

```
void deleteByValue(int key) {
    if (head == null) return;

    Node temp = head;

    // if head itself holds the key
    if (temp.data == key) {
        head = temp.next;
        if (head != null)
            head.prev = null;
        return;
    }

    // search for node to delete
    while (temp != null && temp.data != key)
        temp = temp.next;
```

```
        if (temp == null) return;

        if (temp.next != null)
            temp.next.prev = temp.prev;

        if (temp.prev != null)
            temp.prev.next = temp.next;
    }
}
```

3 Traversal Operations

a. Forward Traversal

java

 Copy code

```
void traverseForward() {
    Node temp = head;
    System.out.print("Forward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
```

b. Backward Traversal

java

 Copy code

```
void traverseBackward() {
    Node temp = head;
    if (temp == null) return;
    while (temp.next != null)
        temp = temp.next;
    System.out.print("Backward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    }
    System.out.println();
}
```

Search Operation

java

 Copy code

```
boolean search(int key) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == key)
            return true;
        temp = temp.next;
    }
    return false;
}
```



Complete Example Code

java

 Copy code

```
class Node {
    int data;
    Node prev, next;

    Node(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

public class DoublyLinkedList {
    Node head;

    void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

```
}

void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.next = newNode;
    newNode.prev = temp;
}

void deleteFromBeginning() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    head = head.next;
    head.prev = null;
}

void deleteFromEnd() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.prev.next = null;
}

void traverseForward() {
    Node temp = head;
    System.out.print("Forward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
}
```

```
    }
    System.out.println();
}

void traverseBackward() {
    Node temp = head;
    if (temp == null) return;
    while (temp.next != null)
        temp = temp.next;
    System.out.print("Backward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    }
    System.out.println();
}

boolean search(int key) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == key)
            return true;
        temp = temp.next;
    }
    return false;
}

public static void main(String[] args) {
    DoublyLinkedList dll = new DoublyLinkedList();

    dll.insertAtBeginning(10);
    dll.insertAtEnd(20);
    dll.insertAtEnd(30);
    dll.insertAtBeginning(5);

    dll.traverseForward();
    dll.traverseBackward();

    System.out.println("Search 20: " + dll.search(20));
    System.out.println("Search 99: " + dll.search(99));

    dll.deleteFromBeginning();
    dll.deleteFromEnd();
}
```



```
        dll.traverseForward();  
    }  
}
```

Key Notes & Recall Points

- ✓ DLL allows **bidirectional traversal** using `prev` and `next` .
- ✓ More **memory** required per node (extra `prev` pointer).
- ✓ **Insertion and deletion** are easier compared to Singly Linked List (no need to track previous node manually).
- ✓ **Head node's prev** is always `null` , and **last node's next** is `null` .
- ✓ Common uses:
 - Undo/Redo functionality.
 - Browser history navigation.
 - LRU (Least Recently Used) Cache implementation.

Would you like me to continue next with **Circular Doubly Linked List (CDLL)** — its working and operations (insert, delete, traverse, etc.)?



Circular Linked List let's now go deep into Doubly Linked List (DLL) in Java, with:

 Complete concepts

 All operations

 Key notes & important recall points

 Example code

Common DSA Interview Questions

Excellent — now we'll go **deep into Circular Doubly Linked List (CDLL)** in Java 💪

This version of the Doubly Linked List connects the **last node back to the first node**, forming a **circular structure** that supports **bidirectional traversal** and **efficient insertion/deletion** at both ends.

💡 Concept of Circular Doubly Linked List (CDLL)

A **Circular Doubly Linked List (CDLL)** is a type of **Doubly Linked List** where:

- The **next** pointer of the **last node** points to the **head**.
- The **prev** pointer of the **head** points to the **last node**.

Thus, you can traverse the list **in both directions endlessly** without hitting a `null`.

🧠 Structure of Node

Each node contains:

1. **data** – stores the element.
2. **prev** – pointer to the previous node.
3. **next** – pointer to the next node.

java

📋 Copy code

```
class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.prev = this.next = null;  
    }  
}
```

⚙️ Core Operations

Let's cover all major operations step by step 🙌

1 Insertion Operations

(a) Insert at the Beginning

Steps:

1. Create a new node.
2. If list empty → make new node point to itself (next and prev).
3. Otherwise:
 - Connect new node between last node and head.
 - Update head pointer.

java

 Copy code

```
void insertAtBeginning(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        head.next = head.prev = head;
        return;
    }
    Node last = head.prev;
    newNode.next = head;
    newNode.prev = last;
    head.prev = newNode;
    last.next = newNode;
    head = newNode;
}
```

(b) Insert at the End

java

 Copy code

```
void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        head.next = head.prev = head;
        return;
    }
    Node last = head.prev;
    newNode.next = head;
```

```
newNode.prev = last;
last.next = newNode;
head.prev = newNode;
}
```

(c) Insert After a Given Node

java

 Copy code

```
void insertAfter(Node prevNode, int data) {
    if (prevNode == null) {
        System.out.println("Previous node cannot be null");
        return;
    }
    Node newNode = new Node(data);
    newNode.next = prevNode.next;
    prevNode.next.prev = newNode;
    prevNode.next = newNode;
    newNode.prev = prevNode;
}
```

2 Deletion Operations

(a) Delete from Beginning

java

 Copy code

```
void deleteFromBeginning() {
    if (head == null) return;

    // If only one node
    if (head.next == head) {
        head = null;
        return;
    }
    Node last = head.prev;
    head = head.next;
    head.prev = last;
    last.next = head;
}
```

(b) Delete from End

java

 Copy code

```
void deleteFromEnd() {
    if (head == null) return;

    if (head.next == head) {
        head = null;
        return;
    }

    Node last = head.prev;
    Node secondLast = last.prev;
    secondLast.next = head;
    head.prev = secondLast;
}
```

(c) Delete by Value

java

 Copy code

```
void deleteByValue(int key) {
    if (head == null) return;

    Node current = head, previous = null;

    // search node with key
    do {
        if (current.data == key)
            break;
        previous = current;
        current = current.next;
    } while (current != head);

    // If node not found
    if (current == head && current.data != key)
        return;

    // Only one node
    if (current.next == head && previous == null) {
```

```
        head = null;
        return;
    }

    // If head node is to be deleted
    if (current == head) {
        Node last = head.prev;
        head = head.next;
        head.prev = last;
        last.next = head;
    } else {
        Node last = head.prev;
        previous.next = current.next;
        current.next.prev = previous;
    }
}
```

3 Traversal

(a) Forward Traversal

java

 Copy code

```
void traverseForward() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    Node temp = head;
    System.out.print("Forward: ");
    do {
        System.out.print(temp.data + " ");
        temp = temp.next;
    } while (temp != head);
    System.out.println();
}
```

(b) Backward Traversal

java

 Copy code

```
void traverseBackward() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    Node last = head.prev;
    Node temp = last;
    System.out.print("Backward: ");
    do {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    } while (temp != last);
    System.out.println();
}
```

Search

java

 Copy code

```
boolean search(int key) {
    if (head == null) return false;
    Node temp = head;
    do {
        if (temp.data == key)
            return true;
        temp = temp.next;
    } while (temp != head);
    return false;
}
```



Complete Example Code

java

 Copy code

```
class Node {
    int data;
    Node prev, next;

    Node(int data) {
```

```
        this.data = data;
        this.prev = this.next = null;
    }
}

public class CircularDoublyLinkedList {
    Node head;

    void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            head.next = head.prev = head;
            return;
        }
        Node last = head.prev;
        newNode.next = head;
        newNode.prev = last;
        head.prev = newNode;
        last.next = newNode;
        head = newNode;
    }

    void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            head.next = head.prev = head;
            return;
        }
        Node last = head.prev;
        newNode.next = head;
        newNode.prev = last;
        last.next = newNode;
        head.prev = newNode;
    }

    void deleteFromBeginning() {
        if (head == null) return;
        if (head.next == head) {
            head = null;
            return;
        }
    }
}
```



```
    Node last = head.prev;
    head = head.next;
    head.prev = last;
    last.next = head;
}

void deleteFromEnd() {
    if (head == null) return;
    if (head.next == head) {
        head = null;
        return;
    }
    Node last = head.prev;
    Node secondLast = last.prev;
    secondLast.next = head;
    head.prev = secondLast;
}

void traverseForward() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    Node temp = head;
    System.out.print("Forward: ");
    do {
        System.out.print(temp.data + " ");
        temp = temp.next;
    } while (temp != head);
    System.out.println();
}

void traverseBackward() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    Node last = head.prev;
    Node temp = last;
    System.out.print("Backward: ");
    do {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    }
```

```
        } while (temp != last);
        System.out.println();
    }

    boolean search(int key) {
        if (head == null) return false;
        Node temp = head;
        do {
            if (temp.data == key)
                return true;
            temp = temp.next;
        } while (temp != head);
        return false;
    }

    public static void main(String[] args) {
        CircularDoublyLinkedList cdll = new CircularDoublyLinkedList();

        cdll.insertAtBeginning(10);
        cdll.insertAtEnd(20);
        cdll.insertAtEnd(30);
        cdll.insertAtBeginning(5);

        cdll.traverseForward();
        cdll.traverseBackward();

        System.out.println("Search 20: " + cdll.search(20));
        System.out.println("Search 99: " + cdll.search(99));

        cdll.deleteFromBeginning();
        cdll.deleteFromEnd();

        cdll.traverseForward();
    }
}
```



Key Notes & Recall Points

- ✓ **Circular nature** – last node connects to head, and head's prev connects to last.
- ✓ Supports **bidirectional traversal**.
- ✓ No node has `null` in its next or prev.

- ✓ Slightly **more complex insertion/deletion** than a normal DLL.
- ✓ **Memory usage** > Singly Linked List.
- ✓ Commonly used in **buffer management, round-robin scheduling, and media playlists**.

Common DSA Interview Questions on DLL / CDLL

1. What is the difference between a singly, doubly, and circular linked list?
2. How do you **reverse a doubly linked list**? (code question)
3. How do you **detect a loop** in a linked list?
4. Write a function to **delete a node with a given key** in a DLL.
5. Explain **real-life use cases** of a circular linked list.
6. Write a program to **insert a node in sorted order** in a DLL.
7. How can you **convert a DLL to a circular DLL**?
8. Find the **middle node** of a doubly linked list.
9. Implement a **Queue/Deque** using a DLL.
10. Compare **time complexities** of insert/delete/search between linked list types.

Would you like me to provide the "Reverse a Doubly Linked List" algorithm next (with explanation and Java code)? It's a **frequent DSA interview question**.



Common DSA coding questions

Excellent ✓ — here's a **complete list of the most common and frequently asked DSA coding questions** (especially around **Linked Lists, Arrays, Strings, Stacks, Queues, Trees, and Graphs**) — perfect for **interviews and coding rounds**.

We'll group them topic-wise so you can practice systematically 🙌

COMMON DSA CODING QUESTIONS (Most Asked in Interviews)

1 Linked List

Singly / Doubly / Circular Linked List

1. Reverse a linked list (iterative & recursive).
 2. Detect a cycle in a linked list (Floyd's cycle detection).
 3. Find the middle of a linked list.
 4. Merge two sorted linked lists.
 5. Remove N-th node from the end of the list.
 6. Find intersection point of two linked lists.
 7. Delete a node without head pointer.
 8. Check if a linked list is palindrome.
 9. Sort a linked list (merge sort).
 10. Add two numbers represented by linked lists.
 11. Flatten a multilevel doubly linked list.
 12. Remove duplicates from a sorted linked list.
 13. Rotate a linked list by K nodes.
 14. Clone a linked list with random pointers.
 15. Convert binary linked list to integer.
-

2 Arrays

1. Find the largest and smallest element in an array.
 2. Reverse an array.
 3. Find the missing number from 1 to n.
 4. Find duplicate elements in an array.
 5. Move all zeros to the end.
 6. Kadane's Algorithm (Maximum Subarray Sum).
 7. Merge two sorted arrays.
 8. Find the intersection and union of two arrays.
 9. Sort an array of 0s, 1s, and 2s (Dutch National Flag problem).
 10. Find subarray with a given sum (sliding window).
 11. Find all pairs with a given sum.
 12. Rotate an array (left or right).
 13. Find majority element ($> n/2$ times).
 14. Find missing and repeating numbers.
 15. Next greater element for each array element.
-



3 Strings

1. Check if a string is palindrome.
 2. Reverse a string (iterative & recursive).
 3. Count vowels, consonants, digits, and spaces.
 4. Check if two strings are anagrams.
 5. Find first non-repeating character.
 6. Longest substring without repeating characters.
 7. Longest palindromic substring.
 8. Implement string rotation check (s1 is rotation of s2).
 9. Find frequency of each character.
 10. Remove duplicates from a string.
 11. Convert string to integer (implement `atoi`).
 12. Implement basic string compression.
 13. Check if two strings are isomorphic.
 14. Print all permutations of a string.
 15. Implement pattern matching (KMP / Rabin-Karp).
-



4 Stack

1. Implement stack using array / linked list.
 2. Implement two stacks in one array.
 3. Evaluate postfix/prefix expressions.
 4. Convert infix to postfix expression.
 5. Next Greater Element using stack.
 6. Check for balanced parentheses.
 7. Sort a stack using recursion.
 8. Get minimum element in stack in $O(1)$ time.
 9. Implement a stack that supports `getMin()`.
 10. Largest rectangle in histogram (hard but common).
-



5 Queue

1. Implement queue using array / linked list.
2. Implement queue using stacks.
3. Implement circular queue.
4. Implement deque (double-ended queue).

5. Reverse a queue using recursion.
 6. Generate binary numbers from 1 to n using a queue.
 7. LRU Cache implementation (using DLL + HashMap).
 8. Rotten Oranges problem (BFS).
 9. Circular Tour (Gas Station problem).
-

6 Tree / Binary Tree / BST

1. Inorder, Preorder, Postorder traversal (recursive & iterative).
 2. Level order traversal (BFS).
 3. Height / depth of a tree.
 4. Count leaf nodes.
 5. Check if two trees are identical.
 6. Find Lowest Common Ancestor (LCA).
 7. Check if a tree is balanced.
 8. Convert sorted array to BST.
 9. Validate Binary Search Tree.
 10. Diameter of a binary tree.
 11. Boundary traversal of a binary tree.
 12. Vertical order traversal.
 13. Mirror a binary tree.
 14. Zig-zag level order traversal.
 15. Path sum problems (find all root-to-leaf paths).
-

7 Graphs

1. Represent graph using adjacency list/matrix.
 2. Depth First Search (DFS).
 3. Breadth First Search (BFS).
 4. Detect cycle in graph (directed/undirected).
 5. Topological Sort.
 6. Shortest Path (Dijkstra's Algorithm).
 7. Minimum Spanning Tree (Prim's / Kruskal's).
 8. Find connected components.
 9. Graph coloring.
 10. Number of islands problem.
-



8 Searching & Sorting

1. Linear Search.
 2. Binary Search (iterative & recursive).
 3. First and last occurrence of element (binary search variant).
 4. Find square root of a number using binary search.
 5. Count number of 1s in a sorted binary array.
 6. Merge Sort.
 7. Quick Sort.
 8. Insertion Sort.
 9. Selection Sort.
 10. Bubble Sort.
-



9 Recursion & Backtracking

1. Factorial of a number.
 2. Fibonacci series using recursion.
 3. Tower of Hanoi.
 4. Print all subsets of a string.
 5. N-Queens problem.
 6. Rat in a Maze.
 7. Sudoku Solver.
 8. Generate all permutations of an array.
 9. Word Search in a matrix.
 10. Subset sum problem.
-



10 Dynamic Programming (DP)

1. Fibonacci using DP.
2. Longest Common Subsequence (LCS).
3. Longest Increasing Subsequence (LIS).
4. 0/1 Knapsack problem.
5. Minimum Edit Distance.
6. Coin Change problem.
7. Matrix Chain Multiplication.
8. Subset sum problem.
9. Rod Cutting problem.

10. Maximum Product Subarray.**1 1 Miscellaneous / Advanced**

1. Implement LRU Cache.
2. Implement LFU Cache.
3. Design a stack that supports getMin().
4. Implement a Trie (prefix tree).
5. Serialize and Deserialize a binary tree.
6. Detect cycle in a directed graph.
7. Find median from data stream.
8. Merge k sorted linked lists.
9. Find intersection of two arrays.
10. Design a Snake and Ladder game (OOP).

**Common Time Complexities (Recall)**

Operation	Singly LL	Doubly LL	Array	Stack	Queue
Insertion at beginning	O(1)	O(1)	O(n)	O(1)	O(1)
Deletion at beginning	O(1)	O(1)	O(n)	O(1)	O(1)
Search	O(n)	O(n)	O(n)	O(n)	O(n)
Access by index	O(n)	O(n)	O(1)	O(n)	O(n)

**Interview Tip**

Focus on mastering:

- Reversal logic
- Cycle detection
- Two-pointer technique
- Hashing and sliding window
- Recursion + backtracking intuition

- Time and space trade-offs
-



Would you like me to give you a "Top 20 Coding Questions to Practice for DSA